

Trabajo Grupal — Análisis e Implementación de Arquitectura Cloud

Caso 1 · Backend Serverless para Aplicación Móvil

1. Ficha general

Caso	01 — Backend Serverless para Aplicación Móvil
Integrantes	4 estudiantes por grupo
Inicio	21 de abril de 2026
Entrega	14 de mayo de 2026 — 11:59 p.m.
Valor	20% de la nota final del curso
Plataforma	Microsoft Azure (Free Tier / Azure for Students)
Formato de entrega	Repositorio GitHub público

2. Arquitectura de referencia Microsoft

Este caso está basado en arquitecturas de referencia oficiales de Microsoft Azure. El grupo debe estudiarlas antes de iniciar el diseño del modelo C4, y los ADRs deben referenciar decisiones concretas que se adapten o aparten de estas guías oficiales.

Arquitectura de referencia — Aplicación Serverless en Azure:

<https://learn.microsoft.com/es-es/azure/architecture/reference-architectures/serverless/web-app>

Documentación de cada servicio del stack:

Azure Functions: <https://learn.microsoft.com/es-es/azure/azure-functions/functions-overview>

Azure API Management: <https://learn.microsoft.com/es-es/azure/api-management/api-management-key-concepts>

Azure Cosmos DB: <https://learn.microsoft.com/es-es/azure/cosmos-db/introduction>

Azure Blob Storage: <https://learn.microsoft.com/es-es/azure/storage/blobs/storage-blobs-introduction>

Azure Notification Hubs: <https://learn.microsoft.com/es-es/azure/notification-hubs/notification-hubs-push-notification-overview>

Los ADRs deben citar decisiones concretas que se aparten o adapten de la arquitectura de referencia oficial, justificando el por qué en el contexto específico de RapidGo.

3. Contexto del caso

3.1 Descripción de la empresa

RapidGo es una startup colombiana de servicios de domicilios fundada en 2022 que opera actualmente en Medellín, Manizales y Pereira. La plataforma conecta a clientes con restaurantes y tiendas locales a través de una aplicación móvil disponible en Android e iOS, desarrollada en React Native, y cuenta con una red de 340 repartidores activos.

En sus primeros dos años de operación, RapidGo procesó en promedio 1.200 pedidos diarios con picos de hasta 4.500 pedidos en días festivos y fines de semana. Su modelo de negocio cobra una comisión del 18% por pedido completado, lo que hace que la disponibilidad del sistema sea directamente proporcional a sus ingresos: cada minuto de caída representa pérdidas estimadas de \$180.000 COP en horas pico.

3.2 Situación tecnológica actual y problemas identificados

El backend actual es una aplicación monolítica en Node.js desplegada en un servidor dedicado en un datacenter de Medellín. El equipo de tecnología ha documentado los siguientes problemas críticos que bloquean el crecimiento de la empresa:

- Escalabilidad manual: en horas pico (12m-2pm y 6pm-9pm) el servidor se satura y el tiempo de respuesta de la API supera los 8 segundos, generando cancelaciones espontáneas de pedidos estimadas en un 12% del tráfico.
- Costo fijo ineficiente: el servidor dedicado cuesta \$4.200.000 COP mensuales independientemente del tráfico. En horas de baja demanda (2am-8am) el uso de CPU no supera el 4%, lo que representa un desperdicio significativo de recursos.
- Despliegues con tiempo de inactividad: cualquier actualización del backend requiere 20-30 minutos de inactividad programada, impactando ventas nocturnas y generando mala experiencia de usuario.

- Notificaciones no confiables: el sistema actual de push notifications tiene una tasa de entrega del 67% debido a la falta de integración directa con FCM y APNs, generando confusión en clientes sobre el estado de sus pedidos.
- Sin tolerancia a fallos: no existe redundancia ni plan de recuperación. Un fallo de hardware implica caída total del servicio con tiempos históricos de restauración de 2 a 6 horas.
- Deuda técnica en autenticación: el manejo de tokens JWT está implementado de forma artesanal en el monolito, sin un gateway centralizado, lo que dificulta agregar nuevos clientes (app web, API pública) en el futuro.

3.3 Requerimientos para la nueva arquitectura

El equipo directivo de RapidGo ha definido los siguientes requerimientos no funcionales que la nueva arquitectura debe cumplir. El grupo debe verificar en los ADRs que las decisiones tomadas satisfacen estos requerimientos:

Requerimiento	Métrica objetivo	Motivación
Disponibilidad	99.9% mensual	Máximo 44 minutos de inactividad al mes
Latencia de API	< 800ms en P95	Reducir cancelaciones por lentitud del sistema
Escalabilidad	500 req/seg sin intervención manual	Soportar días festivos y campañas de marketing
Modelo de costos	Pago por uso real	Eliminar el costo fijo del servidor dedicado
Despliegue	Zero-downtime	No afectar pedidos en curso durante actualizaciones
Notificaciones push	Tasa de entrega > 95%	Mejorar experiencia del cliente con estados en tiempo real

3.4 Restricciones del proyecto

El grupo debe considerar estas restricciones al tomar las decisiones documentadas en los ADRs. Ignorar una restricción sin justificarlo explícitamente en el ADR correspondiente se considera un error de diseño:

- El equipo de desarrollo de RapidGo tiene experiencia en Node.js y Python, pero no en Java ni .NET. Las Functions deben implementarse en uno de estos lenguajes.
- Presupuesto inicial limitado: se deben priorizar servicios con capa gratuita. El gasto mensual en Azure no debe superar los \$50 USD durante la fase piloto.
- La base de datos actual es MySQL relacional con 3 años de datos históricos. Si se propone un cambio de paradigma (relacional a NoSQL), debe estar explícitamente justificado en el ADR-02.
- Los datos de usuarios colombianos deben almacenarse en la región Brazil South o East US por latencia y consideraciones de soberanía de datos.

- La app móvil en React Native no se rediseñará. La nueva API debe mantener compatibilidad con los contratos de endpoints actuales (mismas rutas y estructura de respuesta JSON).
- El equipo de infraestructura es de una sola persona. La solución debe minimizar la carga operativa y evitar servicios que requieran administración manual de servidores o clusters.

4. Stack de servicios Azure

La solución debe contemplar los siguientes cinco servicios. El grupo puede proponer servicios adicionales si los justifica en un ADR propio.

Servicio Azure	Responsabilidad en RapidGo	Tier sugerido
Azure Functions	Lógica de negocio: registrar pedidos, actualizar estados, consultar historial. Escala de 0 a N instancias según demanda.	Consumption Plan (1M ejecuciones/mes gratis)
API Management	Punto de entrada único para la app móvil. Gestiona autenticación JWT, throttling por usuario y versionado de la API.	Developer tier (pruebas y desarrollo)
Cosmos DB	Persistencia de pedidos, usuarios y estados de entrega. Modelo flexible para atributos variables por tipo de negocio.	Free tier (1.000 RU/s, 25 GB incluidos)
Blob Storage	Fotos de comprobantes de entrega, imágenes de productos y exports de reportes operacionales.	LRS Standard (costo mínimo)
Notification Hubs	Notificaciones push a Android (FCM) e iOS (APNs) en tiempo real cuando cambia el estado del pedido.	Free tier (1M notificaciones/mes)

5. Entregables

5.1 Repositorio GitHub

El grupo debe crear un repositorio GitHub público para este proyecto. La documentación completa debe estar consolidada en un único archivo README.md escrito en Markdown. El código fuente va en una carpeta separada.

Requisito obligatorio: la documentación debe estar en el README.md en formato Markdown. El proceso de construcción debe ser visible en el historial de commits. No se aceptan repositorios con un único commit final con todo el trabajo cargado de una vez.

Estructura del repositorio:

Ruta	Contenido esperado
README.md	Documento principal con toda la documentación: portada, C4, ADRs, evidencias de implementación y conclusiones. Escrito en Markdown con imágenes embebidas.
/src/	Código fuente de las Azure Functions, scripts de configuración e infraestructura, y colección de Postman exportada.
/assets/	Imágenes de los diagramas C4 y capturas de pantalla referenciadas desde el README.md.

Etapas mínimas esperadas en el historial de commits:

1. Estructura inicial del repositorio y portada en README.md
2. Diagramas C1 y C2 agregados al README.md
3. Diagrama C3 y sección C4 completa
4. Primeros 2 o 3 ADRs redactados en el README.md
5. Los 5 ADRs finalizados
6. Implementación parcial con primeras evidencias en README.md
7. Entrega final: implementación completa y conclusiones

5.2 Modelo C4

- C1 — Contexto: sistema RapidGo como caja negra, actores (cliente, repartidor, administrador) y sistemas externos (app móvil, pasarela de pagos, FCM, APNs).
- C2 — Contenedores: los cinco servicios Azure, protocolos de comunicación (HTTP/S, SDK de Cosmos, AMQP) y tecnología de cada contenedor.
- C3 — Componentes: interior de Azure Functions con las funciones individuales (registrarPedido, actualizarEstado, consultarHistorial, notificarCliente) y sus dependencias.

Herramienta sugerida: draw.io con la librería de formas C4

5.3 Decisiones arquitectónicas (5 ADRs)

Campo del ADR	Descripción
Título	Decisión tomada en una línea (ej: 'Uso de Azure Functions sobre App Service para la lógica de negocio de RapidGo')
Contexto	Situación específica de RapidGo que motivó la decisión. Debe referenciar requerimientos o restricciones del caso.
Alternativas evaluadas	Mínimo 2 opciones con ventajas y desventajas en el contexto concreto de RapidGo
Decisión	Qué eligieron con justificación técnica y de negocio

Campo del ADR	Descripción
Consecuencias	Ventajas obtenidas y trade-offs asumidos

- ADR-01: Azure Functions vs App Service para la lógica de negocio
- ADR-02: Cosmos DB vs Azure SQL Database para la persistencia de pedidos
- ADR-03: API Management vs exposición directa de las Functions
- ADR-04: Blob Storage vs Azure Files para almacenamiento de archivos
- ADR-05: Notification Hubs vs Azure Communication Services para notificaciones push

5.4 Implementación del flujo crítico

Paso	Acción	Servicio
1	POST /pedidos con datos del pedido en JSON	API Management
2	Recibir, validar y procesar la solicitud	Azure Functions — registrarPedido
3	Guardar el pedido con estado 'confirmado'	Cosmos DB
4	PUT /pedidos/{id} para actualizar estado a 'en camino'	Azure Functions — actualizarEstado
5	Enviar notificación push al cliente con el nuevo estado	Notification Hubs

No se requiere app móvil real. Las llamadas a la API pueden hacerse desde Postman, Thunder Client o curl. Lo que se evalúa es que el flujo funcione de extremo a extremo y que las decisiones de los ADRs sean visibles en la implementación.

Evidencias requeridas (capturas de pantalla documentadas en Markdown):

- Grupo de recursos en Azure con los 5 servicios desplegados
- Logs de ejecución exitosa de las Functions en Azure Portal
- Documento creado en Cosmos DB con la estructura del pedido
- Notificación enviada desde Notification Hubs
- Colección de Postman exportada en /src/ con las llamadas documentadas

6. Rúbrica de evaluación

Componente	Peso	Criterios de evaluación
Modelo C4 (C1, C2, C3)	40% 8 pts	C1 identifica correctamente actores y sistemas externos. C2 muestra todos los servicios con protocolos y tecnologías. C3 desglosa los componentes internos de Functions con sus dependencias. Los tres niveles son coherentes entre sí y con la arquitectura de referencia de Microsoft.

Componente	Peso	Criterios de evaluación
Decisiones arquitectónicas (5 ADRs)	20% 4 pts	Cada ADR sigue la estructura definida. El contexto referencia situaciones concretas de RapidGo. Las alternativas son reales y relevantes en el ecosistema Azure. La justificación es técnica y de negocio. Los trade-offs son honestos y bien argumentados.
Implementación del flujo crítico	40% 8 pts	Los 5 pasos del flujo están implementados y funcionan. Las evidencias están documentadas en Markdown en el repositorio. Lo implementado corresponde con el C4 y los ADRs. El historial de commits muestra construcción progresiva del trabajo.

Criterio transversal — coherencia: un grupo que documente una arquitectura en el C4 y construya algo diferente en la implementación verá afectada la calificación en los tres componentes. Los ADRs deben justificar lo que realmente se implementó.

7. Recomendaciones técnicas

- Activar Azure for Students en portal.azure.com con el correo institucional para acceder a \$100 USD de crédito gratuito.
- Usar la extensión Azure Tools para VS Code para desplegar Functions directamente desde el editor sin usar el portal de Azure.
- Para Notification Hubs: configurar un proyecto Firebase (FCM) gratuito como proveedor Android. No se requiere dispositivo físico; el envío puede simularse desde el portal de Azure.
- Para los diagramas C4: usar draw.io (app.diagrams.net) con la librería de formas C4 disponible en la comunidad.
- Activar el free tier de Cosmos DB al crear la cuenta (una sola cuenta por suscripción de Azure). Si ya está en uso, Azure SQL Database tiene un tier gratuito de 32 GB como alternativa.
- Documentar en Markdown desde el primer día de trabajo. Los commits deben reflejar el avance real del equipo semana a semana, no una carga masiva al final.